

Infraestructura Azure

Registro Horario con IA

Seminario Azure AI Foundry

Este documento describe la configuracion de la infraestructura Azure que da soporte al sistema de registro horario: los App Services, el registro de aplicaciones en Entra ID, los permisos RBAC y el agente de Azure AI Foundry.

App Services API + Webapp en Azure	Entra ID 2 App Registrations	RBAC Permisos AI Foundry	Agente IA Clasico + grok-4.3
--	--	------------------------------------	--

Contenido

1. Azure App Services	3
1.1 Resumen de los dos servicios	3
1.2 Creacion y configuracion con Azure CLI	3
Paso 1 — Crear el grupo de recursos (si no existe)	3
Paso 2 — Crear el App Service Plan	3
Paso 3 — Crear el App Service de la API	3
Paso 4 — Crear el App Service de la Webapp	3
Paso 5 — Activar Oryx build en ambos servicios	3
Paso 6 — Configurar variables de entorno de la Webapp	4
Paso 7 — Despliegue via ZIP deploy	4
1.3 Verificacion del despliegue	4
2. Registro de Aplicaciones en Microsoft Entra ID	6
2.1 App Registration de la API (recurso protegido)	6
Pasos de creacion en el portal de Azure	6
Configuracion del URI de Application ID	6
Nota sobre el formato del token (v1 vs v2)	6
2.2 App Registration de la Webapp (cliente OAuth2)	6
Pasos de creacion en el portal de Azure	7
Agregar la URI de produccion	7
Crear el Client Secret	7
Agregar el permiso delegado de la API	7
2.3 Relacion entre los dos registros	7
3. Permisos RBAC para Azure AI Foundry	8
3.1 Contexto: quien necesita los permisos	8
3.2 Roles asignados	8
3.3 Como asignar los roles (Azure CLI)	8
3.4 Solucion final: API key	9
4. El Agente de Azure AI Foundry	10
4.1 Por que no se pudo usar el nuevo agente de Foundry	10
4.2 El agente clasico: que es y como funciona	10
4.3 Solucion adoptada: Chat Completions con function calling	10
4.4 Creacion del agente clasico	10
Mediante el SDK azure-ai-agents	11
4.5 Cronologia de intentos y errores	11
4.6 Recurso Azure AI Foundry y deployment del modelo	11

1. Azure App Services

El sistema esta compuesto por dos App Services independientes en Azure: uno para la API REST (FastAPI) y otro para la webapp (Flask). Ambos corren en Linux con Python 3.11 y usan el mecanismo de build automatico de Oryx.

1.1 Resumen de los dos servicios

Parametro	API REST (FastAPI)	Webapp (Flask)
Nombre del recurso	ws-dignitae	ws-dignitae-webapp
URL publica	ws-dignitae.azurewebsites.net	ws-dignitae-webapp.azurewebsites.net
Runtime stack	Python 3.11 (Linux)	Python 3.11 (Linux)
Plan de App Service	B1 o superior	B1 o superior
Puerto interno	8000 (Uvicorn/Gunicorn)	5000 (Gunicorn)
Servidor WSGI/ASGI	Gunicorn + Uvicorn workers	Gunicorn
Startup command	gunicorn -w 2 -k uvicorn.workers.UvicornWorker main:app	gunicorn -w 2 app:app

1.2 Creacion y configuracion con Azure CLI

Paso 1 — Crear el grupo de recursos (si no existe)

```
az group create \  
  --name rg-registro-horario \  
  --location westeurope
```

Paso 2 — Crear el App Service Plan

```
az appservice plan create \  
  --name plan-registro-horario \  
  --resource-group rg-registro-horario \  
  --sku B1 \  
  --is-linux
```

Paso 3 — Crear el App Service de la API

```
az webapp create \  
  --name ws-dignitae \  
  --resource-group rg-registro-horario \  
  --plan plan-registro-horario \  
  --runtime "PYTHON:3.11"
```

Paso 4 — Crear el App Service de la Webapp

```
az webapp create \  
  --name ws-dignitae-webapp \  
  --resource-group rg-registro-horario \  
  --plan plan-registro-horario \  
  --runtime "PYTHON:3.11"
```

Paso 5 — Activar Oryx build en ambos servicios

Oryx es el motor de build de Azure que detecta Python, instala las dependencias de requirements.txt y arranca la aplicación automáticamente al hacer un deploy.

```
az webapp config appsettings set \  
  --name ws-dignitae \  
  --resource-group rg-registro-horario \  
  --settings SCM_DO_BUILD_DURING_DEPLOYMENT=true  
  
az webapp config appsettings set \  
  --name ws-dignitae-webapp \  
  --resource-group rg-registro-horario \  
  --settings SCM_DO_BUILD_DURING_DEPLOYMENT=true
```

Paso 6 — Configurar variables de entorno de la Webapp

Las variables que en local están en .env se configuran como Application Settings en Azure:

```
az webapp config appsettings set \  
  --name ws-dignitae-webapp \  
  --resource-group rg-registro-horario \  
  --settings \  
    SECRET_KEY="<valor-aleatorio-largo>" \  
    CLIENT_SECRET="<secreto-del-app-registración>" \  
    REDIRECT_URI="https://ws-dignitae-webapp.azurewebsites.net/callback" \  
    FOUNDRY_API_KEY="<api-key-de-foundry>"
```

Paso 7 — Despliegue via ZIP deploy

Se genera un ZIP con el código de cada aplicación y se sube con el comando de despliegue. Oryx detecta requirements.txt e instala las dependencias automáticamente.

```
# Despliegue de la API  
cd C:\Cursos\seminarioIA\api  
Compress-Archive -Path * -DestinationPath api.zip -Force  
az webapp deploy \  
  --name ws-dignitae \  
  --resource-group rg-registro-horario \  
  --src-path api.zip \  
  --type zip  
  
# Despliegue de la Webapp  
cd C:\Cursos\seminarioIA\webapp  
Compress-Archive -Path * -DestinationPath webapp.zip -Force  
az webapp deploy \  
  --name ws-dignitae-webapp \  
  --resource-group rg-registro-horario \  
  --src-path webapp.zip \  
  --type zip
```

Nota: Startup command automatico

Cuando requirements.txt incluye gunicorn, Oryx lo detecta y genera automáticamente el startup command. Si se necesita personalizar (por ejemplo para la API con Uvicorn workers), se puede configurar en: App Service -> Configuración -> Configuración general -> Startup Command.

1.3 Verificación del despliegue

```
# Ver logs en tiempo real
az webapp log tail --name ws-dignitae --resource-group rg-registro-horario

# Comprobar que la API responde
curl https://ws-dignitae.azurewebsites.net/docs

# Comprobar que la webapp responde
curl https://ws-dignitae-webapp.azurewebsites.net/
```

2. Registro de Aplicaciones en Microsoft Entra ID

El sistema requiere DOS registros de aplicacion en Entra ID: uno para la API (el recurso protegido) y otro para la webapp (el cliente OAuth2). Esta separacion es necesaria porque son dos entidades distintas desde el punto de vista de autentificacion.

Tenant ID	c1f7b851-****-****-****-*****
App Registration API	api-registro-horario — ID: 4dd73579-****-****-****-*****
App Registration Webapp	cliente-registro-horario — ID: c7f7e1c3-****-****-****-*****

2.1 App Registration de la API (recurso protegido)

Este registro representa la API FastAPI. Cualquier cliente que quiera llamar a la API debe obtener un token con el scope de este registro.

Pasos de creacion en el portal de Azure

1. Ir a Entra ID -> Registros de aplicaciones -> Nuevo registro
2. Nombre: api-registro-horario
3. Tipo de cuenta: Solo esta organizacion (single tenant)
4. URI de redireccion: dejar vacia (la API no tiene interfaz de usuario)
5. Pulsar Registrar

Configuracion del URI de Application ID

Por defecto Entra ID asigna un URI del tipo api://<CLIENT_ID>. Este URI es el audience que debe tener el token para que la API lo acepte.

1. En el registro de la API: Exponer una API
2. Establecer URI de ID de aplicacion: api://4dd73579-0a62-438f-ba07-0ebd498c0ec8
3. Agregar un ambito (scope): Agregar un ambito
4. Nombre del ambito: access_as_user
5. Quien puede dar consentimiento: Administradores y usuarios
6. Nombre para mostrar al consentir y descripcion: completar con texto descriptivo
7. Estado: Habilitado -> Guardar

Por que access_as_user

El nombre del scope es arbitrario, pero 'access_as_user' es la convencion habitual para indicar que el cliente actua en nombre del usuario. El scope completo resultante es: api://4dd73579-****-****-****-*****/access_as_user

Nota sobre el formato del token (v1 vs v2)

Por defecto, cuando requestedAccessTokenVersion en el manifiesto de la aplicacion es null o 1, Entra ID emite tokens de formato v1 con:

- issuer: https://sts.windows.net/{TENANT_ID}/
- audience: api://4dd73579-****-****-****-*****

Si requestedAccessTokenVersion se cambia a 2, los tokens son v2 con issuer https://login.microsoftonline.com/{TENANT_ID}/v2.0. En este proyecto usamos el formato v1 (valor por defecto) y la API esta configurada para validarlo en consecuencia.

2.2 App Registration de la Webapp (cliente OAuth2)

Este registro representa la webapp Flask. Es el cliente OAuth2 que inicia el flujo de autenticación y solicita tokens al usuario.

Pasos de creación en el portal de Azure

1. Ir a Entra ID -> Registros de aplicaciones -> Nueva registro
2. Nombre: cliente-registro-horario
3. Tipo de cuenta: Solo esta organización (single tenant)
4. URI de redirección: Web -> `http://localhost:5000/callback` (para desarrollo)
5. Pulsar Registrar

Agregar la URI de producción

1. Ir a Autenticación del registro
2. Agregar URI: `https://ws-dignitae-webapp.azurewebsites.net/callback`
3. Mantener también `http://localhost:5000/callback` para desarrollo
4. En 'Flujos implícitos y flujos híbridos': NO marcar ninguna opción (usamos Authorization Code con PKCE implícito)
5. Guardar

Crear el Client Secret

1. Ir a Certificados y secretos -> Secretos de cliente -> Nuevo secreto de cliente
2. Descripción: webapp-secret, Expiración: 6 meses para la demo (Máximo 2 años)
3. Pulsar Agregar
4. IMPORTANTE: copiar el valor del secreto inmediatamente (solo se muestra una vez)
5. Este valor va en la variable de entorno `CLIENT_SECRET`

Agregar el permiso delegado de la API

1. Ir a Permisos de API -> Agregar un permiso
2. Seleccionar Mis API -> api-registro-horario
3. Tipo: Permisos delegados
4. Marcar: `access_as_user`
5. Pulsar Agregar permisos
6. (Opcional) Conceder consentimiento de administrador para no pedirlo a cada usuario

Por qué permiso delegado y no de aplicación

Un permiso delegado significa que la webapp actúa EN NOMBRE del usuario autenticado. El token resultante contiene la identidad del usuario (UPN, nombre). La API puede extraer ese UPN y filtrar los registros horarios del empleado correcto. Un permiso de aplicación (sin usuario) no permitiría identificar al empleado.

2.3 Relación entre los dos registros

cliente-registro-horario (webapp)	api-registro-horario (API)
Solicita el scope: <code>api://4dd73579-.../access_as_user</code>	Expone el scope: <code>api://4dd73579-.../access_as_user</code>
Usa <code>CLIENT_SECRET</code> para el flujo Authorization Code	Valida el token recibido: <code>audience = api://4dd73579-...</code>
El token resultante identifica al usuario final	Extrae el UPN del token para identificar al empleado

3. Permisos RBAC para Azure AI Foundry

Para que la webapp pueda llamar al modelo grok-4.3 en Azure AI Foundry, la identidad que realiza las llamadas necesita los permisos correctos sobre el recurso de Foundry. Durante el desarrollo se utilizaron permisos RBAC además de la API key.

3.1 Contexto: quien necesita los permisos

En el proyecto final, la webapp se autentica ante Azure AI Foundry usando una API key (variable `FOUNDRY_API_KEY`). Sin embargo, durante el desarrollo intentamos usar el SDK `azure-ai-projects` con autenticación por identidad (`AzureCliCredential`, `ClientSecretCredential`), lo que requirió asignar roles RBAC.

3.2 Roles asignados

Azure AI Developer	Permite crear agentes, threads y ejecutar llamadas en Azure AI Foundry y Azure OpenAI. Incluye permisos de lectura y escritura sobre recursos de IA como agentes y deployments.
Cognitive Services Contributor	Permite acceder a los endpoints de Azure OpenAI (incluido el de grok-4.3). Necesario cuando el recurso OpenAI está ligado al proyecto de Foundry.

3.3 Como asignar los roles (Azure CLI)

Primero obtener el Object ID de la identidad (service principal, usuario o managed identity):

```
# Si se usa un usuario de Azure CLI
az ad signed-in-user show --query id -o tsv

# Si se usa un Service Principal
az ad sp show --id <CLIENT_ID> --query id -o tsv
```

Luego asignar los roles sobre el grupo de recursos o recurso específico:

```
# Obtener el ID del recurso de AI Foundry
az cognitiveservices account show \
  --name ws-reg-horas-resource \
  --resource-group rg-registro-horario \
  --query id -o tsv

# Asignar Azure AI Developer
az role assignment create \
  --assignee <OBJECT_ID> \
  --role "Azure AI Developer" \
  --scope <RESOURCE_ID>

# Asignar Cognitive Services Contributor
az role assignment create \
  --assignee <OBJECT_ID> \
  --role "Cognitive Services Contributor" \
  --scope <RESOURCE_ID>
```

Advertencia: los roles RBAC pueden tardar hasta 5 minutos en propagarse

Si se asigna un rol y se prueba inmediatamente, puede seguir apareciendo el error `PermissionDenied`. Esperar unos minutos y volver a intentarlo antes de diagnosticar otro problema.

3.4 Solucion final: API key

Tras los problemas con el SDK `azure-ai-agents` (incompatibilidades de endpoint, version de API no soportada, formato de agente incorrecto), se optó por autenticacion con API key directamente en el cliente `AzureOpenAI`. Esto simplifica la configuracion y elimina la dependencia de RBAC para la webapp en produccion:

```
# En app settings de Azure App Service:
FOUNDRY_API_KEY = <clave obtenida en Foundry -> Keys and Endpoint>

# En agent_runner.py:
client = AzureOpenAI(
    api_key=os.getenv('FOUNDRY_API_KEY'),
    azure_endpoint='https://ws-reg-horas-resource.openai.azure.com',
    api_version='2025-01-01-preview',
)
```

4. El Agente de Azure AI Foundry

4.1 Por que no se pudo usar el nuevo agente de Foundry

Azure AI Foundry ofrece dos tipos de agentes: el nuevo agente (basado en la Agents API, compatible con AI Studio) y el agente clasico (basado en la OpenAI Assistants API). La distincion es importante porque afecta a la compatibilidad con modelos.

Agente Clasico (OpenAI Assistants API)	Nuevo Agente (Foundry Agents API)
Compatible con cualquier deployment (GPT-*, grok-4.3, etc.)	Solo compatible con modelos GPT-* de OpenAI
SDK: openai (AzureOpenAI) + Chat Completions	SDK: azure-ai-agents o azure-ai-projects
El bucle de herramientas lo implementa el desarrollador	El bucle de herramientas es automatico
Funciona con grok-4.3 en suscripcion MCT	Error: model_not_found con grok-4.3
API version: 2025-01-01-preview	API version especifica del servicio services.ai.azure.com

En esta suscripcion MCT (Microsoft Certified Trainer), los modelos GPT-* de OpenAI no estan disponibles por restricciones de cuota. El unico modelo desplegable es grok-4.3 (xAI). Por tanto, la nueva Agents API de Foundry no es compatible con nuestro modelo.

4.2 El agente clasico: que es y como funciona

El agente clasico de Azure AI Foundry es esencialmente la OpenAI Assistants API alojada en Azure. Permite definir un assistant con instrucciones y herramientas, crear threads (conversaciones) y ejecutar runs (ciclos de razonamiento).

Sin embargo, tambien encontramos incompatibilidad con grok-4.3 al intentar usar el SDK azure-ai-agents para crear threads y runs. El error fue:

```
model_not_found: The requested deployment 'grok-4.3'
cannot be used with this API
```

4.3 Solucion adoptada: Chat Completions con function calling

La solucion fue abandonar completamente los SDKs de agentes (tanto azure-ai-agents como azure-ai-projects) y usar directamente la API de Chat Completions del SDK openai, que si es compatible con grok-4.3. El bucle de razonamiento (ReAct) se implementa manualmente en agent_runner.py.

Funcionalmente equivalente

El resultado es funcionalmente identico a usar la Agents API: el modelo razona, decide llamar herramientas, recibe los resultados y genera una respuesta final. La diferencia es que el desarrollador controla el bucle explicitamente en lugar de delegarlo al servicio.

4.4 Creacion del agente clasico

Se creó uno para investigar la compatibilidad. El portal de Foundry muestra los agentes creados en el portal como 'Agentes clasicos'. Para crear un agente programaticamente via SDK (que si devuelve el ID en formato asst_xxx utilizable por la API):

Mediante el SDK azure-ai-agents

```
from azure.ai.agents import AgentsClient
from azure.identity import AzureCliCredential

client = AgentsClient(
    endpoint='https://ws-reg-horas-resource.services.ai.azure.com',
    credential=AzureCliCredential(),
)

agent = client.create_agent(
    model='grok-4.3',
    name='Asistente Registro Horario',
    instructions='Eres un asistente de registro horario...',
)

print(f'Agent ID: {agent.id}') # -> asst_Ir5fUIo3G4oxmr41GYu0M3ku
```

Importante: el ID del agente del portal NO es el que usa la API

El portal de Azure AI Foundry muestra los agentes con un UUID interno (ej: c548bef5-...). Ese ID NO funciona con la API REST. El ID utilizable tiene formato asst_xxx y solo se obtiene al crear el agente via SDK o via API. En nuestro caso: asst_Ir5fUI*****

4.5 Cronologia de intentos y errores

Para documentar el proceso de aprendizaje, estos fueron los intentos realizados en orden:

Intento 1	AIProjectClient del SDK azure-ai-projects. Error: faltaban parametros endpoint, subscription_id, resource_group_name, project_name en la version 1.0.0b10.
Intento 2	AIProjectClient con todos los parametros. Error: el scope solicitado era https://ml.azure.com/.default pero el endpoint necesitaba https://ai.azure.com/.default.
Intento 3	Wrapper _FoundryCredential para corregir el scope. Error: la version de API 2024-12-01-preview no esta soportada por el endpoint services.ai.azure.com.
Intento 4	Cambio a SDK azure-ai-agents (AgentsClient). Error: PermissionDenied, faltaba el data action Microsoft.CognitiveServices/accounts/AIServices/agents/read.
Intento 5	Asignacion de roles RBAC (Azure AI Developer + Cognitive Services Contributor). Error persistia por tiempo de propagacion.
Intento 6	AgentsClient con permisos correctos. Error: 'AgentsClient' object has no attribute 'create_thread'. Los metodos estaban en sub-clients: client.threads.create(), client.runs.create().
Intento 7	Metodos correctos. Error: model_not_found — grok-4.3 no compatible con Agents API.
Solucion	Cambio a openai.AzureOpenAI con Chat Completions + tool_choice='auto'. Funciona perfectamente con grok-4.3.

4.6 Recurso Azure AI Foundry y deployment del modelo

Nombre del recurso	ws-reg-horas-resource
Endpoint OpenAI	https://ws-reg-horas-resource.openai.azure.com

Modelo desplegado	grok-4.3 (xAI Grok)
Nombre del deployment	grok-4.3
API version usada	2025-01-01-preview
Autenticacion en produccion	API key via variable de entorno FOUNDRY_API_KEY